

INFORME DEL GRUPO 4

Integrantes:

- Gualle Abisag
- Guaman Alexander
- Guanotoa Karla
- Monar Jhair

Fecha: 4/5/2026

Tema: Arreglos

Enunciado: Subarreglos con suma objetivo. Dado un arreglo de 20 números y un valor S : Encuentra todos los subarreglos consecutivos cuya suma sea exactamente S , muestra las posiciones de inicio y fin de cada subarreglo, también evita recalcular sumas desde cero en cada iteración

1. Análisis del problema

Se tiene un arreglo de exactamente 20 números enteros y un valor objetivo S . El objetivo es encontrar todos los **subarreglos consecutivos** (segmentos contiguos del arreglo) cuya suma sea exactamente igual a S .

Un enfoque ingenuo sería calcular la suma de cada subarreglo desde cero, lo cual es ineficiente. En su lugar, el problema pide **evitar recalcular sumas**, lo que sugiere usar técnicas como:

- **Suma acumulada (prefix sum)**, o
- **Ventana deslizante** (si los números son positivos)

La idea es optimizar el cálculo reutilizando resultados previos.

2. Variables de entrada, proceso y salida

Entrada

- $A[20]$: arreglo de 20 números enteros
- S : valor objetivo (entero)

Proceso

- Recorrer el arreglo
- Mantener una suma acumulada
- Usar una estructura (como un mapa/diccionario) para almacenar sumas previas
- Detectar cuándo la suma de un subarreglo es igual a S

Salida

- Lista de subarreglos que cumplen la condición
- Para cada uno:
 - posición inicial
 - posición final

3. Algoritmo

- 1) Leer el arreglo A de 20 elementos
- 2) Leer el valor objetivo S
- 3) Inicializar suma = 0
- 4) Crear un diccionario mapa para guardar sumas acumuladas
- 5) Insertar en el diccionario: mapa[0] = [-1]
- 6) Recorrer el arreglo desde $i = 0$ hasta 19
- 7) En cada iteración, actualizar suma = suma + $A[i]$

- 8) Verificar si existe (suma - S) en el diccionario
- 9) Si existe, entonces:
 - Para cada índice j guardado en mapa[suma - S]
 - Mostrar el subarreglo desde j + 1 hasta i
- 10) Guardar el índice i en mapa[suma]
- 11) Continuar hasta terminar el recorrido del arreglo
- 12) Finalizar el algoritmo

4. Pseudocódigo

Proceso SubarregloSumaObjetivo

Definir arr Como Entero

Dimension arr[20]

Definir S, i, j, suma Como Entero

Escribir "Ingrese 20 numeros:"

Para i <- 1 Hasta 20 Hacer

Leer arr[i]

FinPara

Escribir "Ingrese la suma objetivo:"

Leer S

Para i <- 1 Hasta 20 Hacer

suma <- 0

Para j <- i Hasta 20 Hacer

suma <- suma + arr[j]

Si suma = S Entonces

Escribir "Subarreglo encontrado desde posicion ", i, " hasta ", j

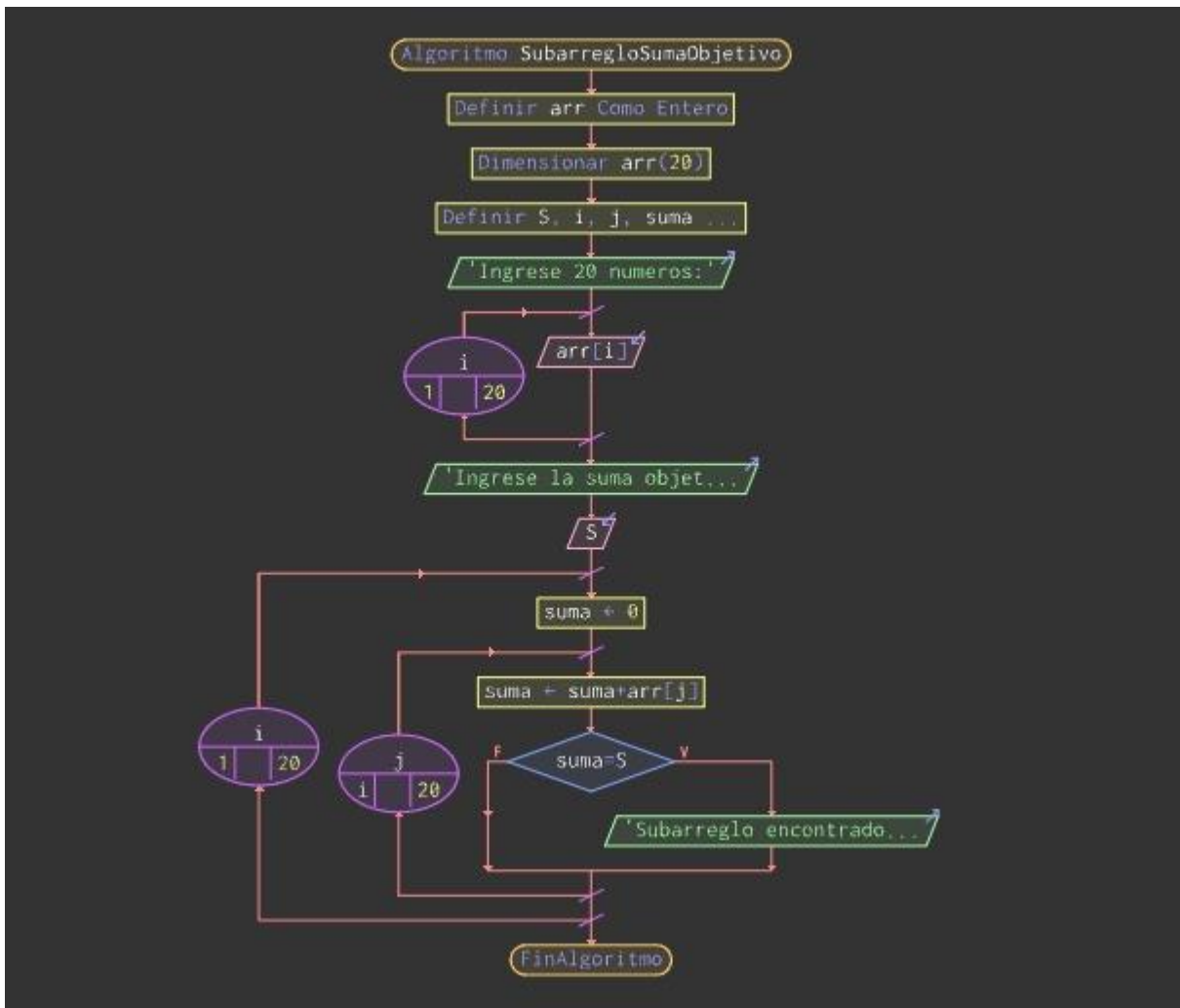
FinSi

FinPara

FinPara

FinProceso

5. Diagrama de flujo



6. Código en C++

```
#include <iostream>
using namespace std;
```

```
int main() {
```

```
    int arr[20];
```

```
    int S;
```

```
    cout << "Ingresar 20 numeros:\n";
```

```
    for (int i = 0; i < 20; i++) {
```

```
        cin >> arr[i];
```

```
    }
```

```
    cout << "Ingresar la suma objetivo: ";
```

```
    cin >> S;
```

```
    for (int i = 0; i < 20; i++) {
```

```

    int suma = 0;

    for (int j = i; j < 20; j++) {
        suma += arr[j];

        if (suma == S) {
            cout << "Subarreglo encontrado desde posicion "
                << i << " hasta " << j << endl;
        }
    }
}

return 0;
}

```

7. Pruebas de solución

```

Ingrese 20 numeros:
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
Ingrese la suma objetivo: 5
Subarreglo encontrado desde posicion 1 hasta 2
Subarreglo encontrado desde posicion 4 hasta 4

```

Se ingresó números del 1 al 20 y una suma objetivo de 5

```
Ingrese 20 numeros:
```

```
2
3
4
5
6
7
8
9
12
23
45
76
87
65
4
3
2
1
3
3
```

```
Ingrese la suma objetivo: 67
```

Se ingresaron números al azar con suma objetivo 67

```
Ingrese 20 numeros:
```

```
20
12
23
45
21
34
5
76
8
9
0
6
5
4
2
3
4
5
6
7
```

```
Ingrese la suma objetivo: 32
```

```
Subarreglo encontrado desde posicion 0 hasta 1
```

```
Subarreglo encontrado desde posicion 8 hasta 13
```

Se ingresaron números al azar con suma objetivo de 32